

EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14-15, 2026

Schedule

1. Aider deep dive
2. Hackathon 1
3. The next assignment
4. Office hours

The harness

THE PROGRAM BETWEEN YOU AND THE MODEL

You have already used four harnesses

	Puts in front of the model	Does with the reply
ChatGPT	Your messages, and notes it kept	Shows it to you
Cursor	Files you point at, and an index	A diff you accept
Claude Code	Files it read on its own, and notes you kept	Edits or runs commands, if allowed
Aider	Files you add, and a map	Writes it, then commits

One kind of model behind all four. The harness is what differs.

A harness is everything around the model



Assemble

Picks every word the model will see.



Answer

The model writes one reply. Text, nothing else.



Apply

Decides what the reply may change, then changes it.

Blue is the harness. Amber is the model.

Take one apart now, build one later.

WHAT A HARNESS IS

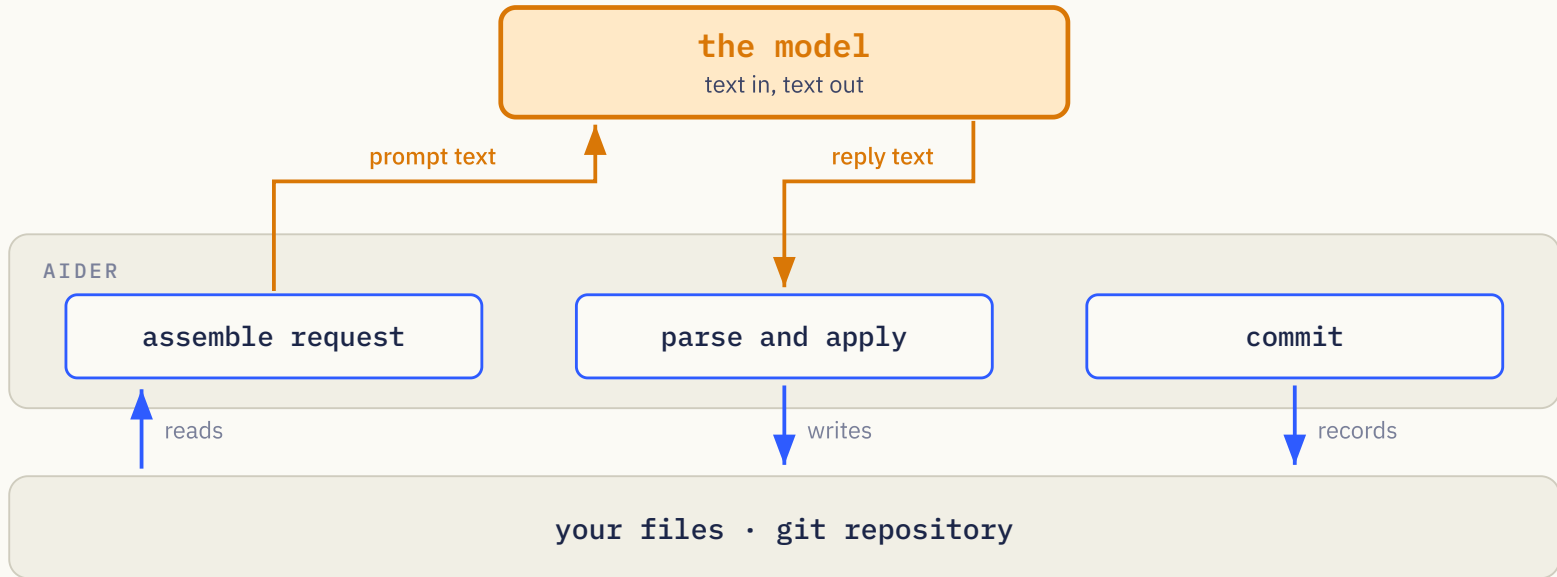
What the model never touches

- Your disk
- Your shell
- Your last session

Text in, text out. The harness does the rest.

AIDER IS ONE

Aider, the model, and the repo



The model touches no file.

Edit-shaped replies

```
path/to/file.py
<<<<<<< SEARCH
the lines exactly as they appear on disk
=====
the lines that replace them
>>>>>>> REPLACE
```

One block, one file. Nothing forces the set to be complete.

Two reply shapes, set per model

	whole	diff
The model returns	The entire file, every time	Search and replace blocks
Fails by	Dropping a line you never mentioned	Not matching the lines on disk
Costs	Every line, every edit	Only the lines that change
Set for	The 4B	The 9B, if you switched it

A harness setting, one line per model. That is why sessions differ.

An action through a text channel

The model returns	The harness needs
Prose	A file path
Prose	An exact region
Prose	The replacement text

Search blocks, tool-call JSON, tagged output. One problem.

Your parser is what turns a reply into an edit, or refuses to.

What Aider does not have

Aider has no	You have seen it in
Editor around it	Cursor
Completions as you type	Cursor
Notes that outlive a session	ChatGPT, Claude Code
Files it went and found itself	Cursor, Claude Code

Every row is a harness feature. None of them is the model.

Context

WHO DECIDES WHAT THE MODEL SEES

Who chooses each part

Part of the prompt	Chosen by
System prompt	Aider, by mode
Added and read-only files	You, by command
Repo map	Aider, by a ranking
History	You start it, Aider trims it
Your message	You

Two rows you never touched. One of them passed for intelligence.

YOUR HALF

What you control

Arranges context

`/add`

`/read-only`

`/drop`

`/clear`

`/tokens`

Changes your files

`/code`

`/undo`

Bare text means `/code`. The default writes.

Most of the interface arranges context. Two commands write.

In Aider, the model can only write

	Aider	Claude Code
The model's verbs	Write	Read, search, edit, run
Wanting a file	Mentions its name in prose	Calls a read tool
Who lets it in	Aider spots the name, you say yes	A permission rule
What lands in the prompt	The file, re-read every turn	The file as it was, once

Everything else Aider does is an add-on around a model that only writes.

It knew about files you never added

```
package/store.py:
:
|class Store:
|    def add(self, title, tags=()) -> Task:
|    def remove(self, task_id: int) -> bool:
:
|    def search(self, text, tag=None) -> list[Task]:
:

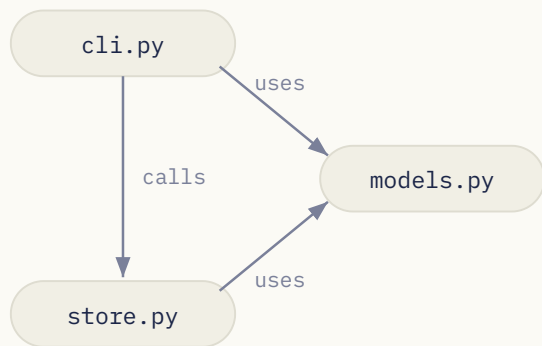
package/models.py:
|@dataclass
|class Task:
:
```

Signatures only.  is what was cut.

This is why it seemed to know your code.

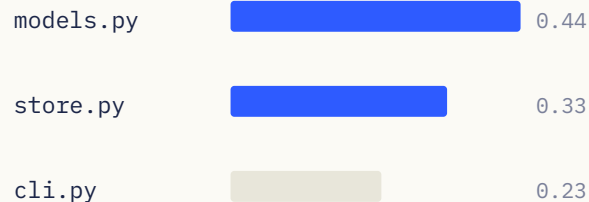
Where the repo map comes from

1. parse every file for what it defines and what it uses



models.py is used by both, so it wins

2. rank by what points at them



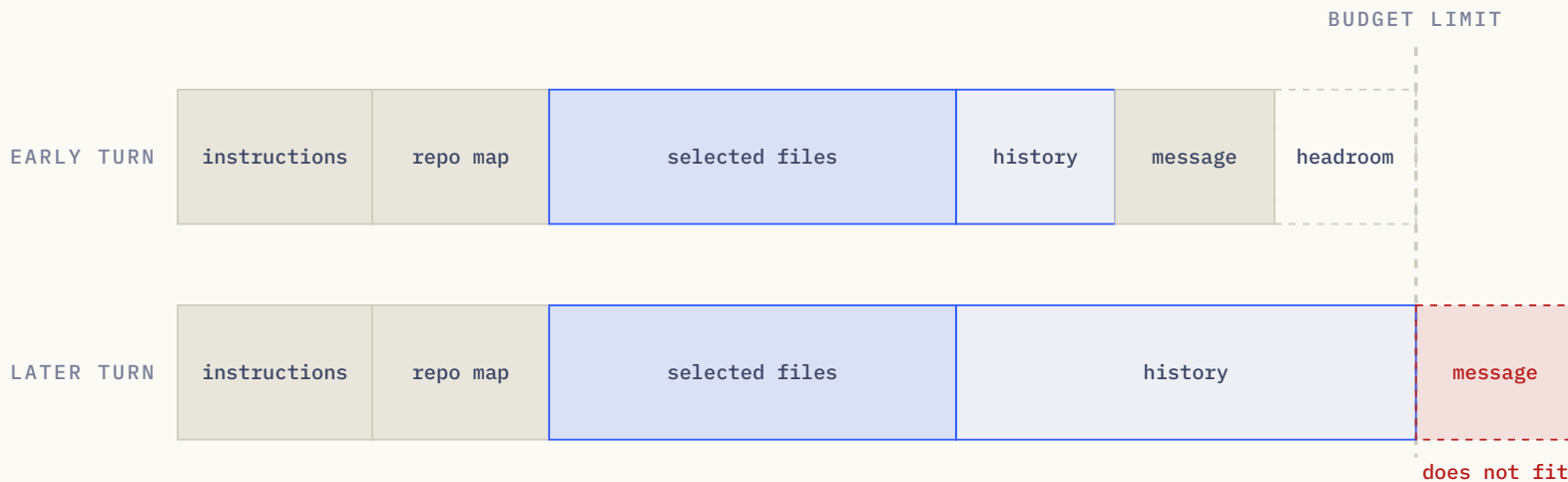
budget ends here

cli.py is left out entirely

PageRank, rendered until the budget ends.

Below the line is invisible, however relevant.

One shared token budget



One budget. Your message is what stops fitting.

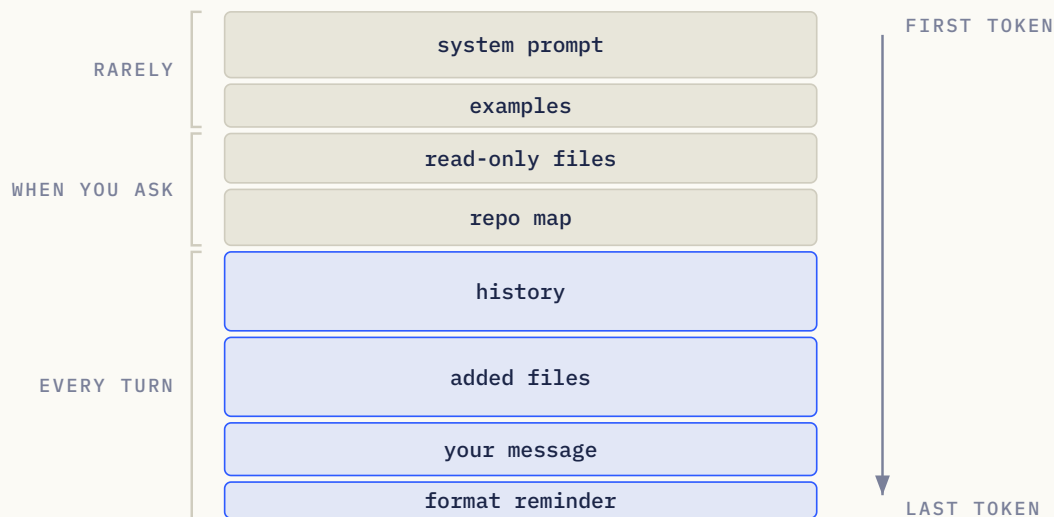
Your build decides what to drop. The packet does not.

Maintaining the prompt

NOTHING CARRIES OVER, SO EVERY PART IS REBUILT EACH TURN

IN ORDER

The prompt, top to bottom



Stable at the top, volatile at the bottom. The order is a design decision.

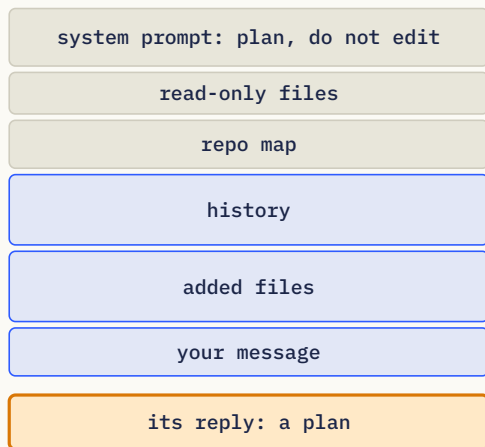
How Aider keeps each part current

Part	Every turn, Aider
System prompt	Swaps it with the mode: ask, code, architect
Read-only files	Re-reads them from disk
Repo map	Re-ranks it around what you just named
History	Adds your message and its reply, summarises when long
Added files	Re-reads them from disk, whole
Your message	Appends a format reminder, dropped when the window is nearly full

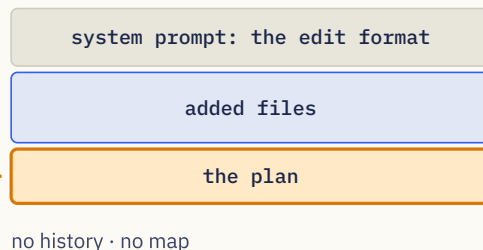
Nothing is remembered. Every row is rebuilt.

Architect mode starts the editor clean

ARCHITECT · THE MODEL YOU TALK TO



EDITOR · A SECOND CALL, STARTED EMPTY



no history · no map

handed over as a message

Two prompts. The second is built fresh: the files and the plan, nothing else.

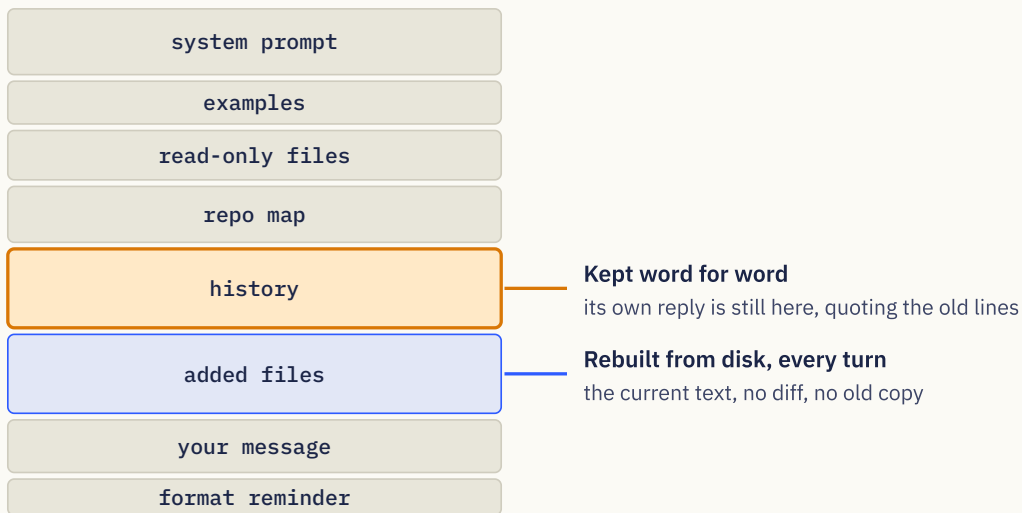
Errors written for the model

Aider answers the model, not you.

- The failed block, quoted back
- Lines from the file that nearly match
- A note if the work already exists

An error message is a prompt.

After an edit lands



Never told what changed. Shown what is true now, and left with what it quoted.

Hackathon 1

ONE FEATURE ON YOUR OWN BUILD, IN ONE EVENING

THURSDAY SEPTEMBER 24

The format

- Two hours, your own project
- Prompt revealed in the room
- Submit before you leave
- Graded separately from the build

The hackathon workspace

- Browser VS Code, already set up
- Aider pointed at the course endpoint
- Your repository already cloned
- A Python reference pack, offline

The pair-programmer

WHAT STAGE 1 ASKS FOR

A harness from scratch

Everything in the last hour, written by you.

It holds	A set of files you chose
It asks	A model endpoint you configure
It writes	Only after you approve

Aider without Aider. Same job, your code.

Due **Tuesday October 6, 11:59 PM.**

Specification before implementation

You write the requirements before you write the code, and the commit history has to show it.

System design	Once, before implementation
Increment specs	Slices you choose, each spec first

No specification is supplied.

The design artifacts

Artifact	Covers
Your requirements	Config, run contract, seven features
Increment specs	Ordered, testable, context scoped
Three diagrams	Components, sequence, edit lifecycle

Specification and design

50 of the 100 build points.

Criterion	Points
Requirements and acceptance criteria	10
Architecture and interface design	10
Increment specs	10
The three diagrams	4 / 3 / 3
Specs before code	5
What changed and why	5

Implementation and tests

50 of the 100 build points.

Criterion	Points
Functional correctness	25
Custom test cases	9
Failure-path tests	8
End-to-end runs and live evidence	5
Operating instructions	3

`RUBRIC.md` carries every row.

WHAT YOU KEEP

The evidence trail

Commits	Specs before the code they describe
Sessions	Which context, which model, what happened
Reconciliation	Where the design changed, and why

Model rules for specifications

- Drafting, critique, diagrams, code, tests
- 9B recommended, 4B permitted
- Secondary models recorded too
- Any compatible host allowed

A different provider is not a different model.

Why this tool, and what's next

AIDER ON PURPOSE, THEN ONE REPOSITORY TO DECEMBER

WHERE IT STARTED

This is where the field started

2023	Aider and Cursor: the model edits your files
2025	Claude Code and Codex: the same loop, most of it automated
Today	Aider still runs the 2023 loop, in source you can open

Start where the field started, then build forward.

Aider is good to learn on

The decision	Where you saw it
Which files the model sees	You typed <code>/add</code>
Which model answers	One line of config
What a reply must look like	The block format
What changed on disk	The diff, then the commit

A tool that decides for you hides what you came to learn.

The models are weak on purpose

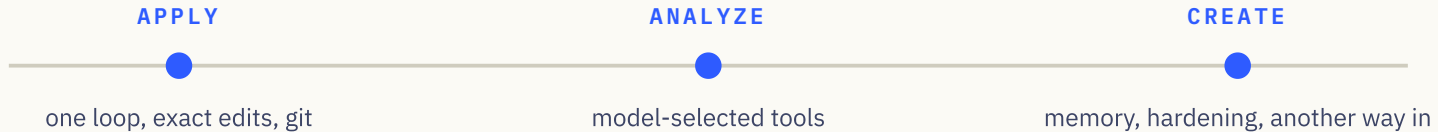
The 4B and the 9B

Free	Your laptop or the course server, no subscription
Fail often	So the diagnosis gets practised every session
Fail for a reason	Context, model or prompt, and you can tell which

When someone says AI is bad, you will know what they saw.

THROUGH DECEMBER

One repository to December



Nothing is thrown away. Each phase grows the last.

You will know how Claude Code works

By December: a coding agent in the shape of Claude Code, grown from the pair-programmer you start this week.

Using it	Knowing it
Approve a permission prompt	Write the rule it enforces
Watch it find the file	Write the search that found it
Type, and the file changes	Write the parser that changed it

Everyone else will use it. You will have written each layer.

EVERY TOOL YOU WILL MEET

The same loop, more machinery

- Assemble, answer, parse, apply, repair, commit
- Every tool you will meet runs it
- What differs is who takes each step

One question is still open: who picks the next action?

Office hours

THE REST OF THE SESSION IS YOURS

Questions?